

API TESTING · POSTMAN · QA AUTOMATION

testHub Postman API Study Guide

A complete, hands-on reference — from microservices architecture to environments, variables, auth flows, pre-request & test scripts, and the full request catalogue of the testHub API collection.

Practice project: testHub — a microservices test-management platform used as a target for API & UI automation practice.

Table of Contents

01	Overview — What This Project Teaches
02	System Architecture (What You're Testing)
03	Postman Setup: Workspace, Environments & Variables
04	Authentication & the JWT Lifecycle
05	The Auto-Refresh Interceptor (Collection Pre-request)
06	Pre-request Scripts — Dynamic Data & Seeding
07	Test Scripts & Assertions (pm.test / chai)
08	Variable Chaining — The End-to-End Workflow
09	Request Catalogue: Health & Auth
10	Request Catalogue: Admin Users
11	Request Catalogue: Projects
12	Request Catalogue: Suites & Test Cases
13	Request Catalogue: Runs & Results
14	Request Catalogue: Dashboard
15	Negative Testing & Status Codes
16	Running at Scale: Collection Runner & Newman
17	Best Practices & Interview Cheat-Sheet

SECTION 01

Overview — What This Project Teaches

This collection was built to **practise real-world API testing** against **testHub**, a small but realistic test-management platform (think a mini TestRail). Rather than hitting a toy endpoint, you exercise a full product: authentication with JWTs, role-based access, paginated lists, nested resources, and aggregated dashboard stats — the same shapes you meet on the job.

Skills demonstrated end-to-end

- Designing a **layered collection** (folders per resource) that mirrors the API.
- **Environments** (Production vs Local) and the difference between collection and environment variables.
- **Auth flows**: register, login, **access + refresh tokens**, Bearer auth, logout/blacklist.
- **Pre-request scripts**: generating unique dynamic data and seeding state with `pm.sendRequest`.
- **Test scripts**: status, schema, value and response-time assertions with chai-style `pm.expect`.
- **Variable chaining**: capturing IDs from one response to drive the next request.
- **Negative testing**: asserting 400/401/403/404 paths, not just happy paths.

- A reusable **JWT auto-refresh interceptor** at collection scope.

■ Interview tip

When asked “how do you structure a Postman collection?” describe exactly this: one folder per resource, requests ordered to form an end-to-end flow, IDs chained through variables, and both positive and negative cases for every endpoint.

SECTION 02

System Architecture (What You're Testing)

testHub is a **microservices** app. Every API call goes through a single **gateway**, which proxies to two backend services sharing one PostgreSQL database. You almost always test through the gateway (baseUrl); the per-service URLs exist mainly to wake or health-check each container.

```
Client (Postman / React SPA)
  | HTTPS (Authorization: Bearer <accessToken>)
  v
[ API GATEWAY ] https://testhub-gateway.onrender.com
  | proxies by path prefix
  |---- /api/auth/*          -> AUTH SERVICE  (:3001)
  |---- /api/projects       -> CORE SERVICE  (:3002)
  |---- /api/suites         -> CORE SERVICE
  |---- /api/testcases      -> CORE SERVICE
  |---- /api/runs           -> CORE SERVICE
  |---- /api/dashboard     -> CORE SERVICE
      |                     |
      v                     v
    PostgreSQL (User)   PostgreSQL (Project/Suite/Case/Run/Result)
```

The three services

Service	Owns	Key responsibility
Gateway	Routing, CORS	Single public entry; proxies by path; serves /health and /docs.
auth-service	User table	Register, login, JWT issue/verify, refresh, logout, admin user mgmt.
core-service	Projects -> Results	Projects, suites, test cases, runs, results, dashboard stats.

Data model (hierarchy): a Project contains Suites; a Suite contains Test Cases; a Run belongs to a Project and produces Results (one per test case, PASS/FAIL/SKIP/BLOCKED). Deleting a project **cascades** to everything under it.

■ Important note — free-tier cold starts

The services sleep after ~15 min idle and take ~24-60s to wake. That's why the collection has **Health / Warm-up** requests and the tests allow response times up to 60s. Always warm up before a run.

SECTION 03

Postman Setup: Workspace, Environments & Variables

The project ships **two environments** so the same requests run against the deployed app or a local stack with a one-click switch — you never edit a request to change targets.

Variable	Production	Local
baseUrl	https://testhub-gateway.onrender.com	http://localhost:3000
authServiceUrl	https://testhub-auth-service.onrender.com	http://localhost:3001
coreServiceUrl	https://testhub-core-service.onrender.com	http://localhost:3002
adminEmail / adminPassword	your admin login	local admin login
accessToken / refreshToken	(set at login)	(set at login)

Collection variables vs Environment variables

Scope	Lives in	Use it for
Collection	The collection itself	Defaults shared by everyone (baseUrl fallback, projectId chaining).
Environment	The active environment	Per-target values & secrets: URLs, credentials, live tokens.
Local (pm.variables)	Single request run	Throwaway values from a pre-request script (e.g. a unique email).

Precedence (narrowest wins): **Local > Environment > Collection > Global**. The collection deliberately writes live tokens to the **environment** scope so a shared collection default never gets polluted with one machine's session.

■ Interview tip

Know the resolution order cold. A classic question: "a variable has different values in the environment and the collection — which wins?" Answer: the **environment** (narrower scope) overrides the collection default.

SECTION 04

Authentication & the JWT Lifecycle

testHub uses **JWT** bearer auth. Login returns two tokens: a short-lived **access token** (~8h) sent on every protected call, and a long-lived **refresh token** (~30d) used only to mint a new access token. The collection sets auth **once at the collection level** so every request inherits it:

```
// Collection > Authorization
Type: Bearer Token
Token: {{accessToken}}

// Login response (POST /api/auth/login) saved by the test script:
pm.environment.set("accessToken", body.accessToken);
pm.environment.set("refreshToken", body.refreshToken);
pm.environment.set("adminUserId", body.user.id);
```

The flow in order

- **Register** → first user becomes ADMIN, everyone after is TESTER (role is server-decided, never trust the client).
- **Login** → receive user + accessToken + refreshToken; persist tokens.
- **Protected calls** → gateway forwards Bearer accessToken; each service verifies it locally.
- **Refresh** → when the access token nears expiry, POST the refresh token for a fresh access token.
- **Logout** → token is blacklisted (Redis) and the client clears its stored tokens.

■ Interview tip — why two tokens?

Short access tokens limit the blast radius if one leaks; the refresh token lets users stay logged in without re-entering credentials. You revoke at logout by blacklisting. Be ready to explain access-vs-refresh in one sentence each.

SECTION 05

The Auto-Refresh Interceptor (Collection Pre-request)

A standout technique in this collection: a **collection-level pre-request script** runs before every request, decodes the JWT, and silently refreshes it if it expires within 60 seconds. This keeps long Collection-Runner sessions authenticated with zero manual steps.

```
// Collection > Pre-request Script (runs before every request)
const token = pm.environment.get("accessToken");
if (token) {
  const [, payloadB64] = token.split(".");
  const payload = JSON.parse(atob(payloadB64)); // decode JWT
  const msLeft = payload.exp * 1000 - Date.now();
  if (msLeft < 60000) { // < 60s to expiry
    pm.sendRequest({
      url: pm.environment.get("baseUrl") + "/api/auth/refresh",
      method: "POST",
      header: { "Content-Type": "application/json" },
      body: { mode: "raw",
        raw: JSON.stringify({ refreshToken:
          pm.environment.get("refreshToken") }) }
    }, (err, res) => {
      if (!err && res.code === 200)
        pm.environment.set("accessToken", res.json().accessToken);
    });
  }
}
```

Key ideas: atob base64-decodes the JWT payload to read exp; pm.sendRequest fires an out-of-band call; the fresh token is written back to the **environment** so the real request that follows uses it.

■ Interview tip

This is a great “show, don’t tell” answer to “how do you handle token expiry in automated runs?” — a pre-request interceptor that refreshes proactively beats letting requests fail with 401 and retrying.

SECTION 06

Pre-request Scripts — Dynamic Data & Seeding

Re-runnable tests must not collide on unique fields like email. The **Register** request generates fresh credentials every run using a timestamp:

```
const t = Date.now();
pm.environment.set("testerEmail", `qa.admin.${t}@testhub.local`);
pm.environment.set("testerPassword", `SecureAdmin!${t}`);
```

The **[Error] Register - Already Exists** request goes further — it **seeds state first**: a pre-request script registers a user via pm.sendRequest, so the main request can then prove a **duplicate** is rejected with 400. This makes the negative test fully self-contained and deterministic.

■ Best practice

Tests should be **idempotent and independent**: generate unique data, seed any preconditions in the pre-request step, and clean up in the test step. Never depend on data a previous manual run left behind.

SECTION 07

Test Scripts & Assertions (pm.test / chai)

Post-response **test scripts** turn a request into an automated check. The collection asserts four things consistently: status code, response schema (keys), specific values, and (for warm-ups) response time.

```
pm.test("Status 200 OK", () => pm.response.to.have.status(200));

pm.test("Response shape is correct", () => {
  const body = pm.response.json();
  pm.expect(body).to.have.all.keys(
    "user", "accessToken", "refreshToken");
  pm.expect(body.user.role).to.eql("ADMIN");
  pm.expect(body.accessToken).to.be.a("string").and.not.empty;
});

// chain: capture an id for the next request
pm.collectionVariables.set("projectId", pm.response.json().id);
```

Assertion	What it proves
pm.response.to.have.status(201)	Correct HTTP status code.
to.have.all.keys(...)	Response has exactly the expected fields (contract / schema).
expect(x).to.eql(y)	A specific value matches (role, status, message).
expect(arr).to.be.an('array')	Type checks on lists and objects.
expect(p).to.be.within(0,100)	Range checks (e.g. passRatePercent).
responseTime).to.be.below(60000)	Performance budget (cold-start tolerant).

■ Interview tip

"What do you assert on an API response?" → status code, body schema, key values, headers/content-type, and response time. Bonus: mention asserting the **error contract** on negative cases, which this collection does.

SECTION 08

Variable Chaining — The End-to-End Workflow

Resources are nested, so each create request **saves its id** and later requests consume it. Running the folders in order produces a complete lifecycle:

```
Login          -> accessToken, refreshToken    (environment)
Create Project  -> projectId                  (collection)
Create Suite    -> suiteId                    (uses projectId)
Create Test Case -> testCaseId                (uses suiteId)
Create Run      -> runId                      (uses projectId)
Submit Result   -> uses runId + testCaseId
Update Run      -> status COMPLETED
Dashboard Stats -> aggregates everything above
Delete Project  -> cascade cleanup; clears ids
```

Recommended run order: Health → Auth → Projects → Suites → Test Cases → Runs → Dashboard. The same ordering is what the Collection Runner / Newman replays in CI.

■ **Important note**

A result's `testCaseId` must belong to the **same project** as the run. Chaining the ids from the create steps guarantees this; pasting a random id will 400/404.

SECTION 09

Request Catalogue: Health & Auth

Health Check (warm-up)

Request	Method & Path	Asserts
Gateway Health	GET {{baseUrl}}/health	200, status 'ok', time < 60s
Auth Health	GET {{authServiceUrl}}/health	200, status 'ok'
Core Health	GET {{coreServiceUrl}}/health	200, status 'ok'

Auth

Request	Method & Path	Notes
Register (New User)	POST /api/auth/register	Dynamic email; expects 201, role TESTER.
Login - Admin	POST /api/auth/login	Saves admin tokens; expects role ADMIN.
Login - New User	POST /api/auth/login	Tester login; saves session tokens.
Get Me	GET /api/auth/me	Current profile from Bearer token.
Refresh Token	POST /api/auth/refresh	New accessToken from refreshToken.
Logout	POST /api/auth/logout	Blacklists token; clears stored tokens.
Re-Login After Logout	POST /api/auth/login	Restores a valid session.

SECTION 10

Request Catalogue: Admin Users

ADMIN-only endpoints — they require the **admin** access token. The list request captures the first TESTER's id into `targetUserId` to drive reset/delete.

Request	Method & Path	Asserts / Behaviour
List All Users	GET /api/auth/users?page=1&limit=10	200; items + pagination; saves targetUserId.
Search Users	GET /api/auth/users?search=...	200; filtered items array.
Reset Password	PATCH /api/auth/users/{{targetUserId}}/reset-password	200; 'Password reset successfully'.
Delete User	DELETE /api/auth/users/{{targetUserId}}	200; can't delete self / last admin.
[Error] List as Tester	GET /api/auth/users	401/403 — role guard rejects TESTER.

■ Interview tip — RBAC

Role-based access control is tested by sending a **TESTER** token to an **ADMIN** endpoint and asserting 403 Forbidden. Always test the *negative* authorization path, not just that admins succeed.

SECTION 11

Request Catalogue: Projects

Request	Method & Path	Asserts / Behaviour
Create Project	POST /api/projects	201; status ACTIVE; saves projectId.
List Projects	GET /api/projects?page&limit	200; items + numeric pagination.
Get Project by ID	GET /api/projects/{{projectId}}	200; correct record; valid status.
Update Project	PUT /api/projects/{{projectId}}	200; new name/description.
Archive Project	PUT /api/projects/{{projectId}}	200; status ARCHIVED.
[Error] Not Found	GET /api/projects/	404; error message present.
Delete Project	DELETE /api/projects/{{projectId}}	200; cascades; clears ids.

Projects are the top of the hierarchy and the ownership anchor — suites, cases and runs all hang off a project, and deleting one cascades to all of them.

SECTION 12

Request Catalogue: Suites & Test Cases

Suites (scoped to a project)

Request	Method & Path	Asserts
Create Suite	POST /api/suites/project/{{projectId}}	201; saves suiteId.
List Suites	GET /api/suites/project/{{projectId}}	200; array of suites.
Update Suite	PUT /api/suites/{{suiteId}}	200; new name.
Delete Suite	DELETE /api/suites/{{suiteId}}	200; cascades to its cases.

Test Cases (scoped to a suite)

Request	Method & Path	Asserts
Create Test Case	POST /api/testcases/suite/{{suiteId}}	201; priority HIGH; saves testCaseId.
Create (Low Priority)	POST /api/testcases/suite/{{suiteId}}	201; priority LOW; empty tags.
List by Suite	GET /api/testcases/suite/{{suiteId}}	200; items + pagination.
Update Test Case	PUT /api/testcases/{{testCaseId}}	200; priority CRITICAL.
Deprecate	PUT /api/testcases/{{testCaseId}}	200; status DEPRECATED.
[Error] Missing title	POST /api/testcases/suite/{{suiteId}}	400; validation errors[].
Delete Test Case	DELETE /api/testcases/{{testCaseId}}	200; success message.

Priority values: LOW, MEDIUM, HIGH, CRITICAL. Status values: ACTIVE, DEPRECATED. Note there is no GET-by-id for suites or cases — you list by parent.

SECTION 13

Request Catalogue: Runs & Results

Request	Method & Path	Asserts / Behaviour
Create Run	POST /api/runs/project/{{projectId}}	201; status IN_PROGRESS; saves runId.
Create Run w/ TC ids	POST /api/runs/project/{{projectId}}	201; run scoped to selectedCaseIds.
List Runs	GET /api/runs/project/{{projectId}}	200; array of runs.
Get Run w/ Results	GET /api/runs/{{runId}}	200; results[] + summary counts.
Submit Result (PASS)	POST /api/runs/{{runId}}/results	200; upsert; status PASS.
Submit Result (FAIL)	POST /api/runs/{{runId}}/results	200; same case -> FAIL (upsert).
[Error] Invalid Status	POST /api/runs/{{runId}}/results	400; status must be enum.
Update Run (Complete)	PUT /api/runs/{{runId}}	200; status COMPLETED.
Delete Run	DELETE /api/runs/{{runId}}	200; success message.

Upsert behaviour: submitting a result for the same (runId, testCaseId) pair updates the existing result rather than creating a duplicate — that's why PASS then FAIL on the same case is a valid demonstration. Result statuses: PASS, FAIL, SKIP, BLOCKED. Run statuses: IN_PROGRESS, COMPLETED, ABORTED.

SECTION 14

Request Catalogue: Dashboard

Request	Method & Path	Asserts
Get Dashboard Stats	GET /api/dashboard/stats	200; all stat keys; passRate 0-100; activeProjects<=total.
[Error] No Token	GET /api/dashboard/stats	401; error message present.

The stats payload aggregates the whole estate: totalProjects, activeProjects, totalTestCases, totalRuns, activeRuns, passRatePercent, recentRuns (max 5), resultBreakdown {PASS,FAIL,SKIP,BLOCKED}, latestRunName/Status/Results.

■ Good assertions to show off

Beyond “is it 200”, this request asserts **business invariants**: pass rate is within 0-100 and activeProjects can never exceed totalProjects. Asserting invariants, not just types, is what separates a senior test from a junior one.

SECTION 15

Negative Testing & Status Codes

Roughly a third of the collection is **negative tests** — the cases that prove the API fails *correctly*. Each asserts both the status code and the error body contract.

Code	Meaning	Example in this collection
400	Bad Request / validation	Empty login fields; missing test-case title; invalid result status.
401	Unauthorized	Get Me with no token; wrong password; invalid refresh token; dashboard no token.
403	Forbidden (role)	TESTER token hitting an ADMIN-only users endpoint.
404	Not Found	Get project with a non-existent id.

Code	Meaning	Example in this collection
201	Created	Successful create of project / suite / case / run.
200	OK	Reads, updates, logins, result submissions.

■ Interview tip

“Why test error paths?” Because most production incidents are the unhappy path: bad input, missing auth, wrong role. A suite that only tests 200s gives false confidence.

SECTION 16

Running at Scale: Collection Runner & Newman

Because requests are ordered and chained, the whole collection runs unattended in the **Collection Runner** (GUI) or **Newman** (CLI / CI). Newman is the same engine, scriptable for pipelines:

```
# install once
npm install -g newman newman-reporter-htmlextra

# run the collection against the Production environment
newman run "testHub API - Full Collection.postman_collection.json" \
  -e "testHub - Production.postman_environment.json" \
  --reporters cli,htmlextra \
  --reporter-htmlextra-export reports/testhub.html
```

In CI you'd warm the services first (hit /health), then run Newman and publish the HTML report as a build artifact. The collection's ordering + variable chaining is exactly what makes this reproducible.

SECTION 17

Best Practices & Interview Cheat-Sheet

What this collection does right

- **One folder per resource**, requests ordered into an end-to-end flow.
- **Environments** for Prod/Local; secrets & tokens in environment scope, not hard-coded.
- **Collection-level auth** so every request inherits the Bearer token (DRY).
- **Collection pre-request interceptor** that auto-refreshes the JWT.
- **Dynamic, unique data** in pre-request scripts so runs are repeatable.
- **Self-seeding negative tests** (register-then-duplicate) for determinism.
- **Variable chaining** of ids through the whole lifecycle.
- **Positive and negative** coverage with status + schema + value assertions.
- **Cleanup** on delete (cascade + clearing of chained ids).

Rapid-fire interview answers

Question	One-line answer
Collection vs environment variable?	Env is per-target & narrower; it overrides the collection default.
Pre-request vs test script?	Pre-request runs before the call (setup); test runs after (assertions).
How to chain requests?	Save a value with <code>pm.*.set()</code> in tests; read it as <code>{{var}}</code> in the next request.
Access vs refresh token?	Access = short-lived, sent every call; refresh = long-lived, mints new access.

Question	One-line answer
How to handle token expiry?	A pre-request interceptor decodes exp and refreshes proactively.
What do you assert?	Status, schema/keys, key values, headers, response time, error contract.
How to run in CI?	Newman with the environment file; publish an HTML report artifact.
Why negative tests?	Most real failures are the unhappy path — bad input, auth, roles.

Run order to memorise: Health → Auth → Projects → Suites → Test Cases → Runs → Dashboard. Warm up first; clean up last.

■ Final note

If you can walk an interviewer through this collection — architecture, environments, the auth lifecycle, the refresh interceptor, chaining, and your assertion strategy — you've demonstrated practical, job-ready API testing skill, not just theory.